



Politécnico
Internacional

ESTRUCTURA DE DATOS

“Celebra cada pequeña **victoria.**”

AGENDA

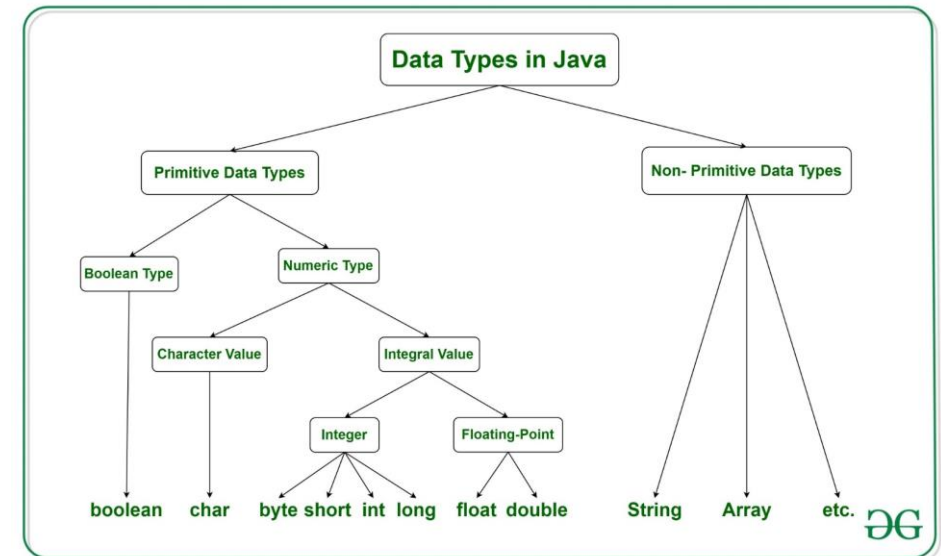
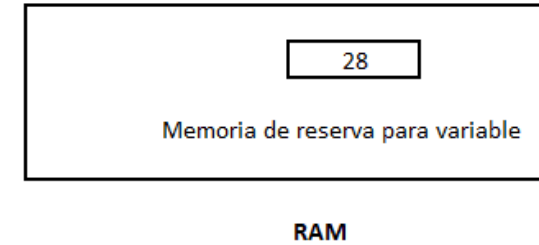
- Conceptos Java
- Innova Colombia
- Conclusiones

- ¿Qué es Java?
- Características – Seguridad
- POO
- Sintaxis Básica
- Historia
- Tipado – Estático & Dinámico
- Tipos de datos.
- Operadores
- Variables
- Lectura de Datos
- Estructuras Condicionales
- Bucles (While + For + For Each + Do While)
- Comentarios
- Arreglos
- Listas Enlazadas
- Estructura de Datos
- Arreglos Bidimensionales

Retroalimentación

`int age = 28;`

valor
tipo de dato
nombre_variable





Politécnico
Internacional



Retroalimentación

ARREGLOS Y MATRICES



<https://www.youtube.com/watch?v=W8QwbhynEmc&t=267s>



Politécnico
Internacional



https://www.youtube.com/watch?v=mEvjuG_tbqY

Tutorial Java - 9. Arreglos

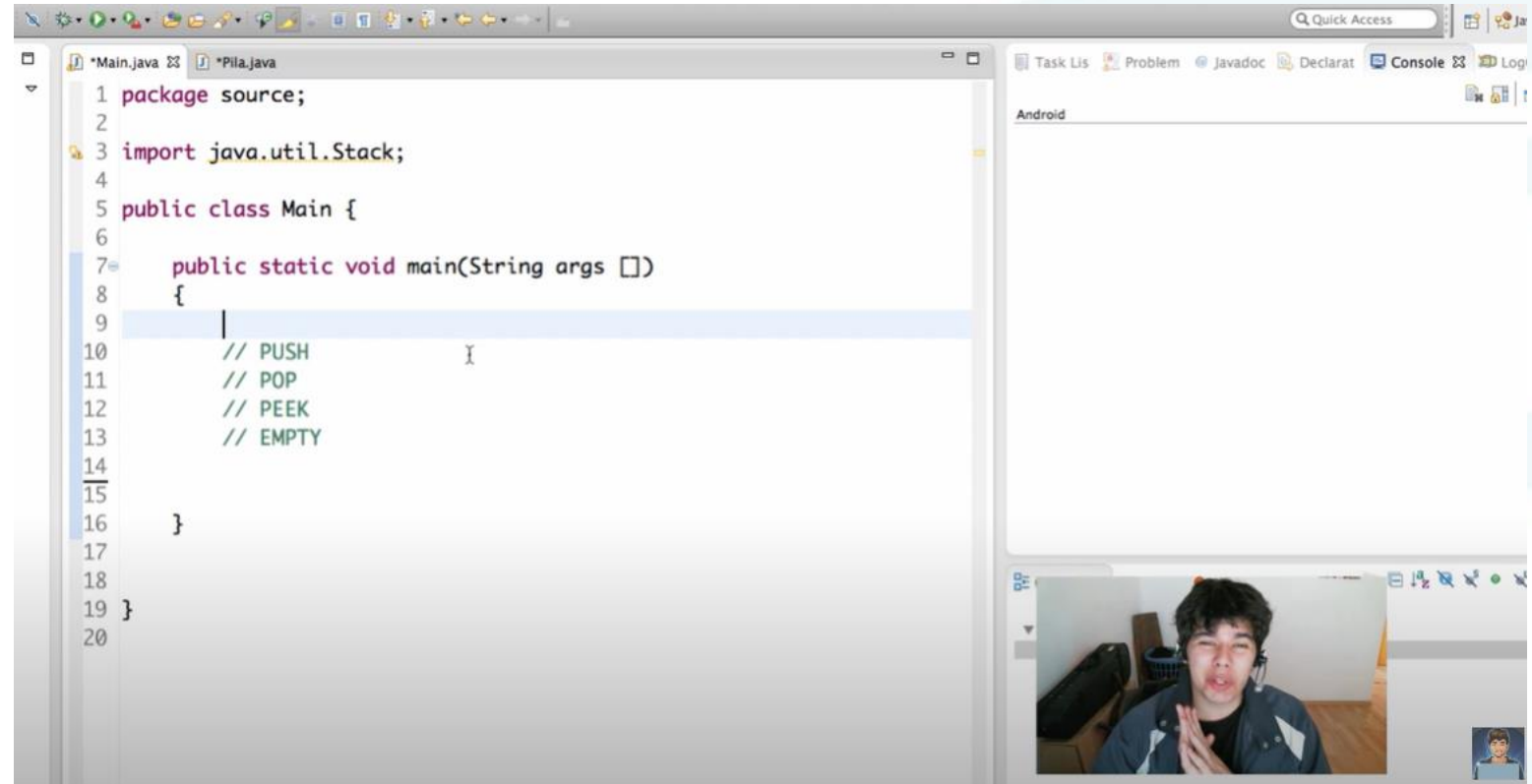


Politécnico
Internacional



Capítulo 6. Listas Enlazadas URL

https://www.youtube.com/watch?v=r_ttiJ45mfw



```
1 package source;
2
3 import java.util.Stack;
4
5 public class Main {
6
7     public static void main(String args [])
8     {
9         |
10        // PUSH
11        // POP
12        // PEEK
13        // EMPTY
14
15
16    }
17
18 }
19
20
```

Estructura de datos en Java

<https://www.youtube.com/watch?v=BdgdjTFpC6g>



Politécnico
Internacional



Retroalimentación



Arreglos bidimensionales (Matrices dinámicas)

<https://www.youtube.com/watch?v=3FIfrC1kG0>



Politécnico
Internacional



AVANCES **INNOVA** COLOMBIA



Politécnico
Internacional



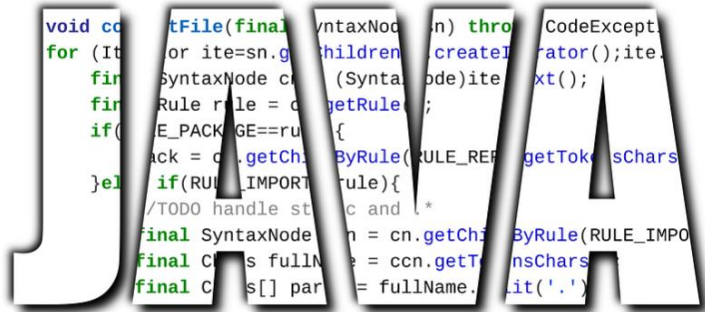
CLASE SCANNER



Politécnico
Internacional



*Scanner es una clase en el Paquete **JAVA.UTIL** utilizada para obtener la entrada de los tipos primitivos como int, double etc. y también **STRING**. Es la forma más fácil de leer datos en un programa Java, aunque no es muy eficiente si se quiere un método de entrada para escenarios con validación.*



```
void createFile(final SyntaxNode n) throws IOException {
    for (Iterator<SyntaxNode> ite = n.getChildren().iterator(); ite.hasNext(); ) {
        SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.getPackageName() != null) {
            final String pack = child.getCharByRule(RULE_REFERENCE, getTokensChars());
            if (rule.isImportRule()) {
                // TODO handle static and *
                final SyntaxNode n = child.getCharByRule(RULE_IMPORT, getTokensChars());
                final Class fullN = ccn.getTokensChars();
                final Class[] pars = fullN.getMethods();
            }
        }
    }
}
```

IMPORTANTE

Para crear un objeto de clase Scanner, normalmente pasamos el objeto predefinido **System.in**, que representa el flujo de entrada estándar. Podemos pasar un objeto de clase File si queremos leer la entrada de un archivo.

Para leer valores numéricos de un determinado tipo de datos XYZ, la función que se utilizará es **Nextxyz()**. Por ejemplo, para leer un valor de tipo short, podemos usar **Nextshort()**.

Para leer cadenas (**STRINGS**), usamos **Nextline()**.

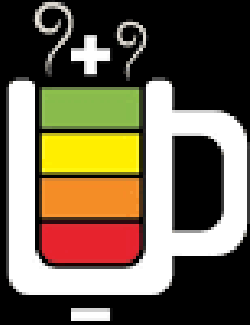
Para leer un solo carácter, se usa **next().charAt(0)**. La función **next()** devuelve el siguiente token/palabra en la entrada como cadena y la función **charAt (0)** devuelve el primer carácter de esa cadena.



Politécnico
Internacional



CÓDIGO



```
...ava CIDE_CICLO_DO_WHILE.java x CIDE_VISUAL_PERI_TRI_EQ.java x CIDE_ENTRADA_SCANNER.java x
Source History
35 long mobileNo = sc.nextLong();
36 double average = sc.nextDouble();
37
38 // Imprima los valores para verificar si la entrada
39 // fue obtenida correctamente.
40 System.out.println("Nombre Estudiante CIDE: "+name);
41 System.out.println("Género Estudiante CIDE: "+gender);
42 System.out.println("Edad Estudiante CIDE: "+age);
43 System.out.println("Teléfono Estudiante CIDE: "+mobileNo);
44 System.out.println("Promedio Estudiante CIDE: "+average);
45
46
47 }
48
cide_entrada_scanner.CIDE_ENTRADA_SCANNER > main >
Output - CIDE_ENTRADA_SCANNER (run) x
run:
harol
masculino
35
22222
5
Nombre Estudiante CIDE: harol
Género Estudiante CIDE: m
Edad Estudiante CIDE: 35
Teléfono Estudiante CIDE: 22222
Promedio Estudiante CIDE: 5.0
BUILD SUCCESSFUL (total time: 24 seconds)
```



HASNEXTXYZ()

A veces, debemos verificar si el siguiente valor que leemos es de cierto tipo o si la entrada ha finalizado (se encontró el marcador EOF).

*Luego, se verifica si la entrada del Scanner es del tipo que queremos con la ayuda de las funciones **hasNextXYZ()** donde **XYZ** es del tipo que nos interesa. La función devuelve true (verdadero) si el scanner tiene un **token** de ese tipo, de lo contrario es false (falso).*

*En el siguiente código, se utiliza **hasNextInt()**. Para verificar una cadena, usamos **hasNextLine()**. De manera similar, para verificar si hay un solo carácter, se usa **hasNext().CharAt(0)**.*



```
Start Page x CIDE_ENTRADA_SCANNER.java x CIDE_ENTRADA_SCANNER_HASTNETXYZ.java x
Source History
10  * @author Harol
11  */
12  // CIDE ENTRADA DE DATOS
13  // FUNDAMENTOS DE PROGRAMACION
14  public class CIDE_ENTRADA_SCANNER_HASTNETXYZ {
15
16      /**
17       * @param args the command line arguments
18       */
19      public static void main(String[] args) {
20          // TODO code application logic here
21
22          // Declarar un objeto e inicializar con
23          // el objeto de entrada estándar predefinido
24          Scanner sc = new Scanner(System.in);
25
26          // Inicializar la suma y el recuento
27          // de los elementos de entrada
28          int sum = 0, count = 0;
29
Output - CIDE_ENTRADA_SCANNER_HASTNETXYZ (run) x
run:
1
2
3.0
La media es: 1
BUILD SUCCESSFUL (total time: 3 seconds)
```



Politécnico
Internacional



SINTAXIS BÁSICA DE JAVA



Todos los archivos pertenecen a un paquete

Importa los paquetes para el proyecto



Java usa clases para ejecutar el código

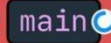


Se debe indicar el tipo de dato.

Modificadores de acceso: private, public, protected o por defecto ninguno.



El método principal en Java es el método **main**



La palabra reservada **new** crea un objeto del tipo de dato especificado.



Se utilizan ; para cada sentencia



Se usan llaves para identificar el bloque de código



```
package team.ed.course;

import java.lang.*;

public class Person {
    private String name;

    public static void main(String args[]){
        Person friend = new Person();
        friend.name = "Peter";
        System.out.println("Hola " +
            friend.name);
    }
}
```



Aprende Java desde cero hasta nivel Jedi en:

 ed.team/java

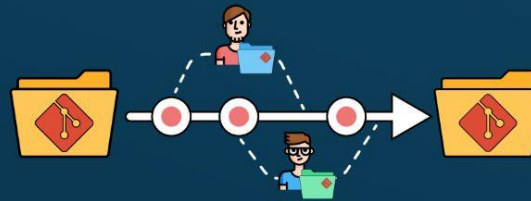




¿SABES QUÉ ES GIT?



Es un sistema de **control de versiones**. Lleva un registro de todos los **cambios y avances** de tu proyecto.



● GIT TRABAJA CON RAMAS

Ayuda a que **varias personas trabajen** en un mismo proyecto y pueden realizar **modificaciones sin afectar a los demás** archivos. Una vez que estén listos los cambios se **fusionan con la rama principal**.

Prof. Beto Quiroga



Todo desarrollador sin importar el lenguaje **debe dominar Git**.

v1 — v2 — v3

● Funciona como una **máquina de tiempo**, puedes ir al pasado de tu código o volver al presente.



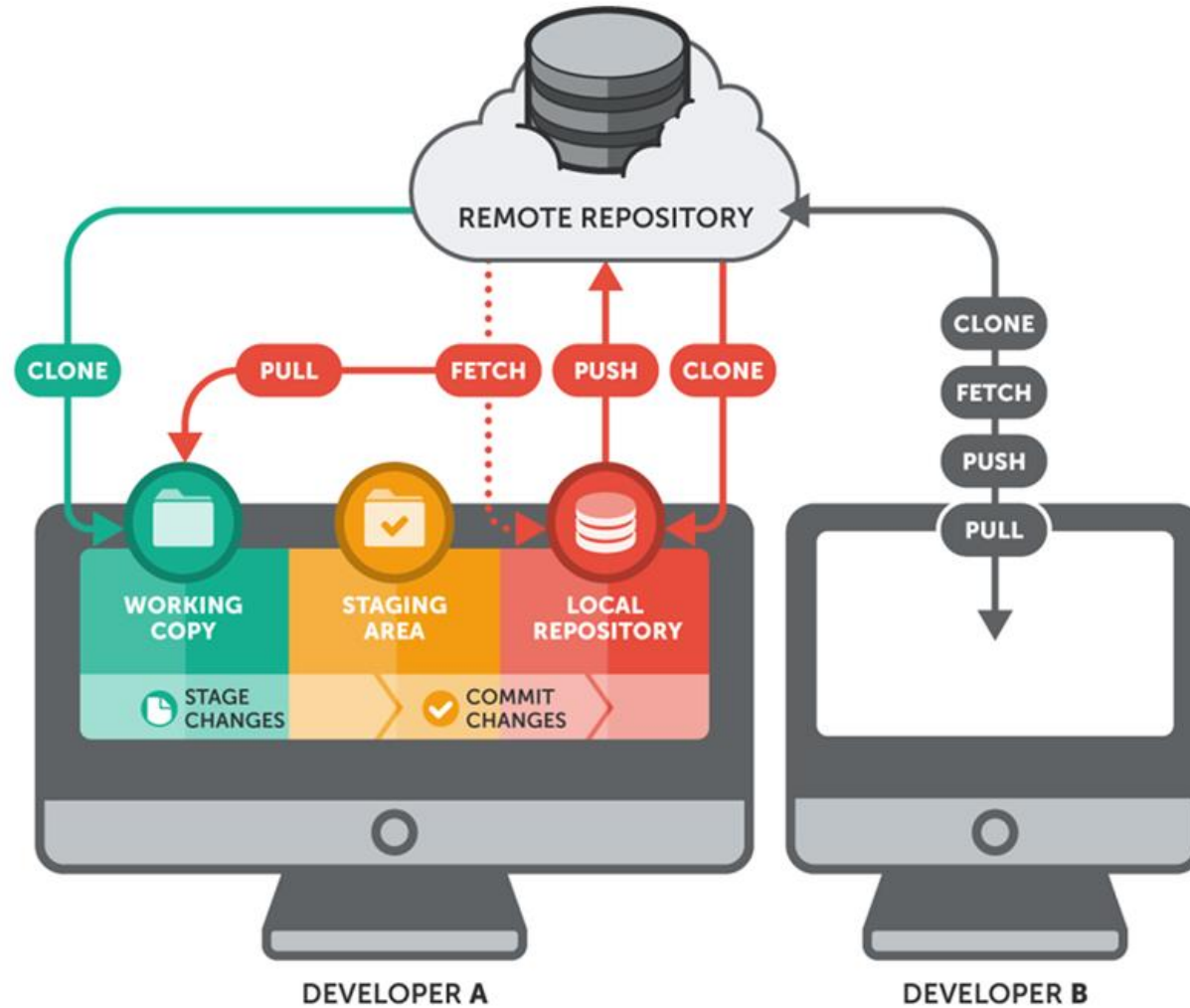
● **GitHub** es un servicio que te ayuda a **almacenar tu proyecto en la nube**, además existen otros servicios como **Gitlab** o **Bitbucket**.

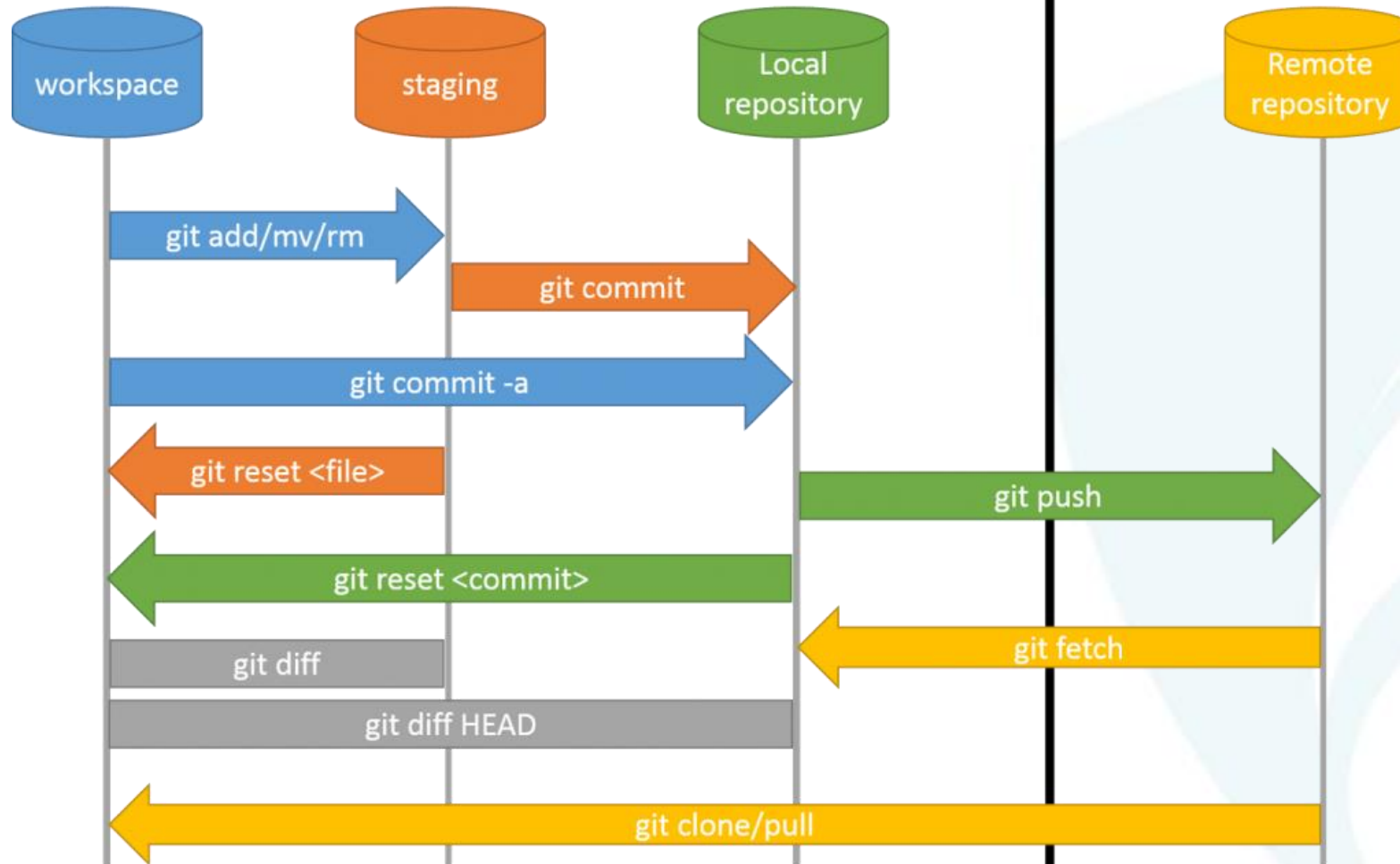


Aprende a trabajar en equipo con Git en:

 ed.team/cursos/git-workflow









COMANDOS BÁSICOS DE git



GIT es un sistema de control de versiones que nos ayuda a llevar el historial completo de modificaciones de un proyecto.

Todo desarrollador sin importar el lenguaje **debe dominar Git**.
Prof. Beto Quiroga

GIT INIT



Inicia un nuevo repositorio.

GIT ADD



Añade un archivo a la zona de **montaje**. `git add *` añade uno o más archivos a la zona de montaje.

GIT LOG



Se utiliza para listar el **historial** de versiones de la rama actual.

GIT RESET



Descompone el archivo, pero conserva el contenido del mismo.



GIT CLONE

Clona un repositorio existente.

GIT CONFIG



Establece el nombre del autor, el correo y demás **parámetros** que Git utiliza por defecto.

GIT STATUS



Enumera todos los archivos que deben ser confirmados.

GIT DIFF



Muestra las **diferencias** de archivo que aún no se ponen en escena.

¿Qué comandos faltó? Haremos la 2da parte con tus sugerencias

 ed.team/cursos/git



COMANDOS BÁSICOS DE git (PARTE 2)

Si quieres trabajar en el mundo del desarrollo de software, es obligatorio que sepas GIT.

GIT HELP



Muestra información para saber el uso de cada comando de git.

GIT CLEAN



Elimina archivos no deseados de un repositorio.

GIT BRANCH



Muestra la lista de ramas que existen en un repositorio.

GIT MERGE



Fusiona dos o más ramas.

GIT PULL



Descarga y actualiza los cambios realizados desde un repositorio remoto a tu repositorio local.

GIT PUSH



Sube los archivos a un repositorio remoto.

GIT CHECKOUT



Sirve para moverse de una rama a otra y regresar en el tiempo.

GIT REMOTE



Crea, visualiza y elimina conexiones a otros repositorios.

Si eres programador es **obligatorio** aprender GIT. Domínalo en:

 ed.team/cursos/git



TÉRMINOS DE git QUE DEBES CONOCER

REPOSITORY/REPO

Base de datos donde se almacena el historial del código.

COMMIT

Registro de uno o varios cambios hechos en el repositorio.

STAGE

Lista de archivos que se usarán para un commit.

FORK

Copia de un repositorio.

BRANCH / RAMA

Entorno o espacio de trabajo independiente en Git.

MASTER

Rama principal de un repositorio.

CHECKOUT

Es la acción de **moverse entre diferentes ramas.**

MERGE

Combina dos o más ramas en una.

Si eres programador es **obligatorio** aprender GIT. Domínalo en:

 ed.team/cursos/git



EL MENSAJE DE COMMIT **IDEAL EN GIT**

```
$ git commit
```

1

Crea un nuevo componente para el countdown

2

* Reemplaza el compontente que generó conflictos

3

* Visible solo para usuarios premium

4

Resuelve issue: #193"

1 TÍTULO



Resumen breve de los cambios realizados.

No debes usar más de 50 caracteres.



2 CUERPO (OPCIONAL)



Explicación más detallada que aporta contexto e información adicional.

3



Puedes usar **varias frases o crear listas para explicar**

4



Menciones de los issues afectados o relacionados.

El mensaje del commit **explica la razón detrás de los cambios.**



Beto Quiroga
COO EDteam



Encuentra más consejos como este y domina Git en:

 ed.team/cursos/git





GITHUB VS GITLAB



● **Máximo de 3** colaboradores gratis.



● Ofrece **hasta 500 MB** gratis por repositorio.



● El código se puede **descargar** como archivo.



● Opción de **lectura o escritura por repositorio**.



● Supera los **35 millones** proyectos.



● Fue comprada por Microsoft ¿**Bueno o malo?** ¿Qué opinas?



● **No hay límite de** colaboradores.



● El límite gratis por repositorio es de **10 GB**.



● Permite **incluir adjuntos en un issue** o problema.



● Puedes **proporcionar acceso** a determinadas partes de un proyecto.



● Más de **100 mil** proyectos.

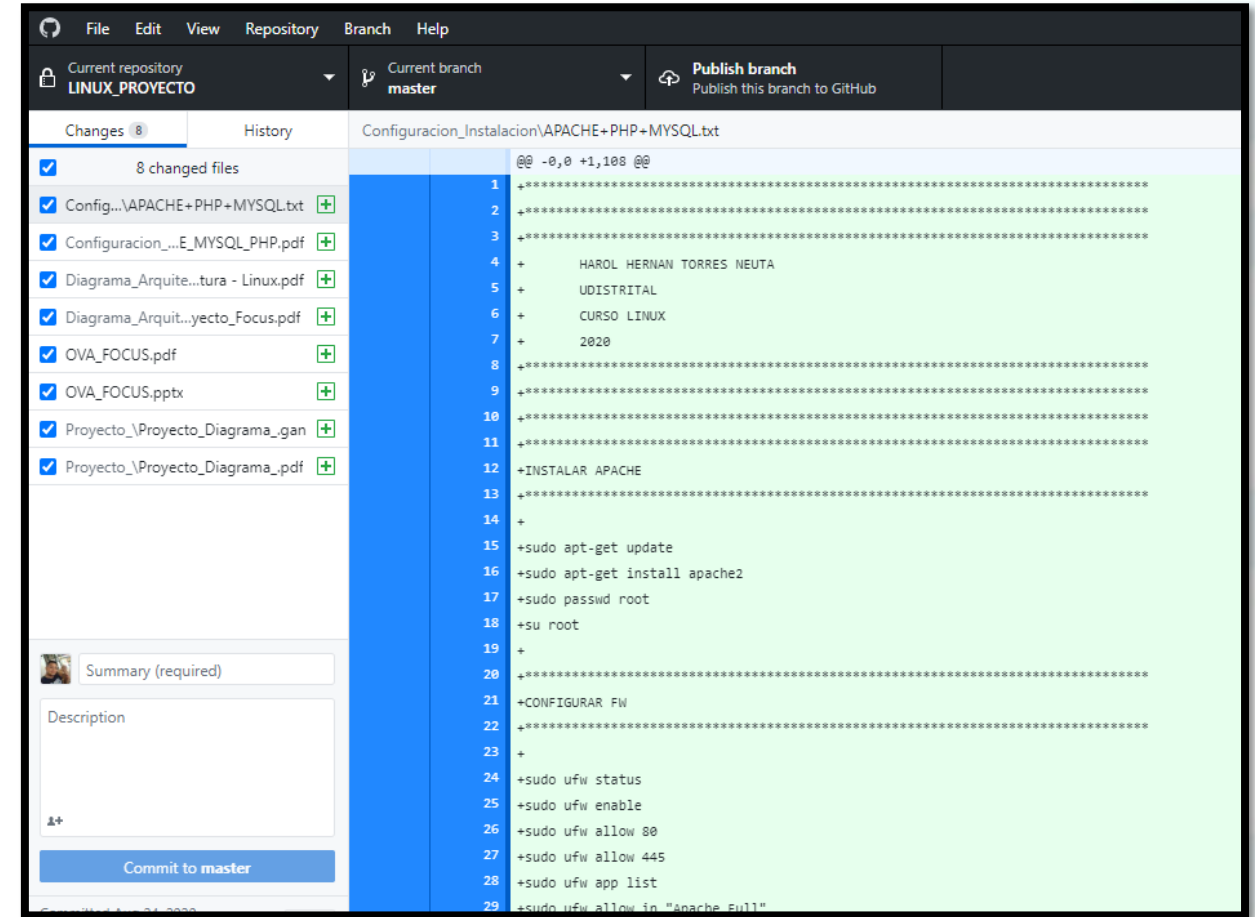


● Opera bajo **licencia de código abierto**.

Aprende gestionar tus repositorios en GitHub y GitLab en:

`>-` ed.team/cursos/git-workflow







En Java, **null** está asociado a **java.lang.NullPointerException**. Como es una clase en el paquete **java.lang**, se invoca cuando intentamos realizar algunas operaciones con o sin null y, a veces, ni siquiera sabemos dónde ha sucedido.

*null es literal en Java y porque las palabras clave distinguen entre mayúsculas y minúsculas en java, no podemos escribir **NULL** o 0*

Código NULL

```
public class Test
{
    public static void main (String[] args) throws java.lang.Exception
    {
        // error en tiempo de compilación: no se puede encontrar el símbolo 'NULL'
        Object obj = NULL;

        //lo siguiente se ejecuta con éxito
        Object obj1 = null;
    }
}
```

Salida

```
error: cannot find symbol
  can't find symbol 'NULL'
                        ^
  variable NULL
  class Test
1 error
```

Código NULL

```
public class Test
{
    private static Object obj;
    public static void main(String args[])
    {
        // imprimira nulo;
        System.out.println("El valor del objeto obj es : " + obj);
    }
}
```

Salida

```
El valor del objeto obj es : null
```

Valor de variable de referencia

Cualquier variable de referencia en Java tiene un valor predeterminado nulo.

Código NULL

```
// null se puede asignar a String
String str = null;

// tambien puedes asignar null a Integer
Integer itr = null;

// null también se puede asignar a Double
Double dbl = null;

//null se puede castear a String
String myStr = (String) null;

// también puede ser casteado a Integer
Integer myItr = (Integer) null;

// si es posible, no hay error
Double myDbl = (Double) null;
```

TIPO DE NULL

*A diferencia de la idea errónea común, **null no es un objeto ni un tipo**. Es solo un valor especial, que se puede asignar a cualquier tipo de referencia y se puede castear null a cualquier tipo.*

Código NULL

```
public class Test
{
    public static void main (String[] args) throws java.lang.Exception
    {
        //Un número entero puede ser nulo, así que esto está bien
        Integer i = null;

        //Unboxing null a integer lanza NullPointerException
        int a = i;
    }
}
```

Salida

```
Exception in thread "main" java.lang.NullPointerException
at Test.main
```

Autoboxing y unboxing

Durante las operaciones de auto-boxing y unboxing, el compilador simplemente arroja un error de excepción NullPointerException si se asigna un valor nulo al tipo de datos en recuadro primitivo.

Código NULL

```
public class Test
{
    public static void main (String[] args) throws java.lang.Exception
    {
        Integer i = null;
        Integer j = 10;

        //imprime false
        System.out.println(i instanceof Integer);

        //Compila con éxito
        System.out.println(j instanceof Integer);
    }
}
```

Salida

```
false
true
```

Operador Instanceof

El operador ***instanceof*** de Java se usa para probar si el objeto es una instancia del tipo especificado (clase, subclase o interfaz). En tiempo de ejecución, el resultado del operador ***instanceof*** es verdadero si el valor de la expresión no es nulo.

Esta es una propiedad importante de la operación ***instanceof***, que lo hace útil para las verificaciones de conversión de tipo.

Código NULL

```
public class Test
{
    public static void main(String args[])
    {
        Test obj= null;
        obj.staticMethod();
        obj.nonStaticMethod();
    }

    private static void staticMethod()
    {
        //Puede ser llamado por referencia nula
        System.out.println("metodo estatico, puede ser llamado por referencia nula");
    }

    private void nonStaticMethod()
    {
        //Can not be called by null reference
        System.out.print(" Metodo no estatico- ");
        System.out.println("no puede ser llamado por referencia nula");
    }
}
```

Salida

```
Exception in thread "main" java.lang.NullPointerException
  at Test.main
metodo estatico, puede ser llamado por referencia nula
```

Métodos estáticos vs no estáticos

No podemos llamar a un método no estático en una variable de referencia con valor nulo, arrojará NullPointerException, pero podemos llamar al método estático con variables de referencia con valores nulos.

Como los métodos estáticos están vinculados mediante el enlace estático, no lanzarán NullPointerException.



Un **ARRAY** o matriz es simplemente una variable que puede contener valores múltiples, a diferencia de una variable regular que solo puede contener un único valor.



Los siguientes son algunos puntos importantes acerca de las matrices, arreglos o **ARRAYS** de Java.



1. En Java, todas las matrices se asignan dinámicamente.
2. Como las matrices/arrays son objetos en Java, cada array tiene asociado una variable de instancia de longitud (length) que contiene la cantidad de elementos que la matriz puede contener. (En otras palabras, length contiene el tamaño de la matriz.)
3. Una variable array en Java se declara como otras variables con corchetes [] después del tipo de datos.
4. Las variables en el array están ordenadas y cada una tiene un índice que comienza desde 0.



5. *El array Java también se puede usar como un campo estático, una variable local o un parámetro de método.*
6. *El tamaño de un array debe especificarse mediante un valor int y no, long o short.*
7. *La superclase directa de un tipo de array es Object.*
8. *Cada tipo de array implementa las interfaces Cloneable y java.io.Serializable.*
9. *El array puede contener tipos de datos primitivos así como también objetos de una clase según la definición del array. En el caso de los tipos de datos primitivos, los valores reales se almacenan en ubicaciones de memoria contigua. En el caso de los objetos de una clase, los objetos reales se almacenan en heap.*



Un **ARRAY** o arreglo es una colección de variables del mismo tipo, a la que se hace referencia por un nombre común. En Java, los arrays pueden tener una o más dimensiones, aunque el array unidimensional es el más común.

Los arrays se usan para una variedad de propósitos porque ofrecen un medio conveniente de agrupar variables relacionadas. Por ejemplo, puede usar una matriz para mantener un registro de la temperatura alta diaria durante un mes, una lista de promedios de precios de acciones o una lista de tu colección de libros de programación.



La ventaja principal de un **ARRAY** es que organiza los datos de tal manera que puede ser manipulado fácilmente. Por ejemplo, si tiene un array que contiene los ingresos de un grupo seleccionado de hogares, es fácil calcular el ingreso promedio haciendo un ciclo a través del array. Además, los arrays organizan los datos de tal manera que se pueden ordenar fácilmente. Aunque los arrays en Java se pueden usar como matrices en otros lenguajes de programación, tienen un atributo especial: se implementan como objetos.

Este hecho es una de las razones por las que la discusión de los arrays se pospuso hasta que se introdujeron los objetos. Al implementar arrays como objetos, se obtienen varias ventajas importantes, una de las cuales es que los arrays no utilizados pueden ser recolectados.

```
tipo nombre-array[];
```

```
tipo [] nombre-array;
```

ARRAYS UNIDIMENSIONALES

Un array unidimensional es una lista de variables relacionadas. Tales listas son comunes en la programación. Por ejemplo, puede usar un array unidimensional para almacenar los números de cuenta de los usuarios activos en una red.

Código

```
// ambas son declaraciones válidas
int intArray[];
int[] intArray;
//Tipo de datos primitivos
byte byteArray[];
short shortArray[];
boolean booleanArray[];
long longArray[];
float floatArray[];
double doubleArray[];
char charArray[];
//Tipos de datos definidos por el usuario
// una serie de referencias a objetos de
// la clase MyClass (una clase creada por
// el usuario)
MyClass myClassArray[];
Object[] ao,          // array de Object
Collection[] ca;    // array de Collection
```

LA DECLARACIÓN DE UN ARRAY TIENE DOS COMPONENTES: EL TIPO Y EL NOMBRE.

tipo declara el tipo de elemento del ARRAY. El tipo de elemento determina el tipo de datos de cada elemento que comprende la matriz. Al igual que la matriz de tipo int, también podemos crear una matriz de otros tipos de datos primitivos como char, float, double..etc o tipo de datos definido por el usuario (objetos de una clase). Por lo tanto, el tipo de elemento para la matriz determina el tipo de datos que la matriz contendrá.

```
nombre-array = new tipo ;
```

1. *tipo especifica el tipo de datos que se asignará*
2. *tamaño especifica el número de elementos en el array*
3. *nombre-array es el nombre de la variable del array vinculado al mismo.*
4. *Es decir, para usar new para asignar un array, debe especificar el tipo y la cantidad de elementos a asignar.*

INSTANCIANDO UN ARRAY EN JAVA

Cuando un array se declara, solo se crea una referencia del array. Para realmente crear o dar memoria al array (a partir de aquí solo mencionaré a array, y no matriz o arreglo).

```
int intArray[]; //declarando un array  
intArray = new int; // asignando memoria al array
```

```
int[] intArray = new int; // combinando ambas declaraciones en una
```

Código

```
int[] intArray = new int[]{ 1,2,3,4,5,6,7,8,9,10 };  
// Declarando un array literal
```

Array Literal

- *En una situación en la que ya se conoce el tamaño y los elementos del array, se pueden usar literales del array.*
- *La longitud de este array determina la longitud del array creado.*
- *No es necesario escribir `new int[]` en las últimas versiones de Java*

Código

```
// Programa Java para ilustrar la creación de un array de enteros,
// coloca algunos valores en la matriz, e imprime cada valor

class DemoArray
{
    public static void main (String[] args)
    {
        // declara un array de enteros.
        int[] arr;

        // asignando memoria para 5 enteros.
        arr = new int[5];

        // inicializa el primer elemento del array
        arr[0] = 10;

        // inicializa el segundo elemento del array
        arr[1] = 20;

        // y así...
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        // accediendo a los elementos del array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Elemento en el índice " + i + "
                               " : "+ arr[i]);
    }
}
```

```
//acceder a los elementos del array
for (int i = 0; i < arr.length; i++)
    System.out.println("Elemento en el índice " + i + " : "+ arr);
```

ACCEDIENDO A LOS ELEMENTOS DEL ARRAY USANDO EL BUCLE FOR

A cada elemento del array se accede a través de su índice. El índice comienza con 0 y termina en (tamaño total del array) -1. Se puede acceder a todos los elementos de la matriz usando el [bucle for en Java](#).

Salida

```
Elemento en el índice 0 : 10
Elemento en el índice 1 : 20
Elemento en el índice 2 : 30
Elemento en el índice 3 : 40
Elemento en el índice 4 : 50
```

```
Student[] arr = new Student; //student es una clase definida por el usuario
```

```
// Programa Java para ilustrar la creación de
// un array de objetos

class Student
{
    public int roll_no;
    public String name;
    Student(int roll_no, String name)
    {
        this.roll_no = roll_no;
        this.name = name;
    }
}

// Los elementos del array son objetos de la clase Student
public class DemoArray2
{
    public static void main (String[] args)
    {
        // declara una array de enteros.
        Student[] arr;

        // asigna memoria para 5 objetos del tipo Student.
        arr = new Student[5];

        // inicializa el primer elemento del array
        arr[0] = new Student(1,"uno");

        // inicializa el segundo elemento del array
        arr[1] = new Student(2,"dos");

        // y así...
        arr[2] = new Student(3,"tres");
        arr[3] = new Student(4,"cuatro");
        arr[4] = new Student(5,"cinco");

        // accediendo a los elementos del array
        for (int i = 0; i < arr.length; i++)
            System.out.println("Elemento en " + i + " : " +
                               arr[i].roll_no + " " + arr[i].name);
    }
}
```

Código

ARRAYS DE OBJETOS

Se crea una matriz de objetos como una matriz de elementos de datos de tipo primitivo.

El array Student contiene siete espacios de memoria, cada uno del tamaño de la clase Student, en los que se puede almacenar la dirección de siete objetos de Student. Los objetos de Student deben crearse con el constructor de la clase Student.

Salida

```
Elemento en 0 : 1 uno
Elemento en 1 : 2 dos
Elemento en 2 : 3 tres
Elemento en 3 : 4 cuatro
Elemento en 4 : 5 cinco
```

Código

```
class multiDimensional
{
    public static void main(String args[])
    {
        // declarar e inicializar array 2D
        int arr[][] = { {2,7,9},{3,6,1},{7,4,2} };

        // imprimir array 2D
        for (int i=0; i< 3 ; i++)
        {
            for (int j=0; j < 3 ; j++)
                System.out.print(arr[i][j] + " ");

            System.out.println();
        }
    }
}
```

```
int[][] intArray = new int; //un array 2D o matrix
int[][][] intArray = new int; //una array 3D
```

ARRAYS MULTIDIMENSIONALES

Las matrices multidimensionales son matrices de matrices o arrays de arrays, donde cada elemento del array contiene la referencia de otro array. Se crea una matriz multidimensional al agregar un conjunto de corchetes ([]) por dimensión.

Salida

```
2 7 9
3 6 1
7 4 2
```

Código

```
// Programa Java para demostrar
// el uso de array como parámetro

class Test
{
    // Método principal
    public static void main(String args[])
    {
        int arr[] = {3, 1, 2, 5, 4};

        // pasar array al método sum
        sum(arr);

    }

    public static void sum(int[] arr)
    {
        // obtener suma de valores del array
        int sum = 0;

        for (int i = 0; i < arr.length; i++)
            sum+=arr[i];

        System.out.println("Suma de valores del array: " + sum);
    }
}
```

PASAR ARRAYS A MÉTODOS

Al igual que las variables, también podemos pasar arrays a los métodos. Por ejemplo, en el programa siguiente se pasa un array al método sum para calcular la suma de los valores del array

Salida

Suma de valores del array: 15

Capítulo 13

Interfaces gráficas de usuario (GUI)



Curso Java desde cero | Interfaces gráficas

<https://www.youtube.com/watch?v=nxhZbBbRoSI>



CrissUD / ProyectoFrontendJava Public

Watch 1 Fork 2 Star 21

Code Issues Pull requests Actions Projects Wiki Security Insights

master 1 branch 0 tags

Go to file Add file Code

CrissUD Se realizan actualizaciones de servicios para ingreso de usuario 9e00046 on 30 Sep 2020 43 commits

ProjectoInterfazGrafica	Se realizan actualizaciones de servicios para ingreso de usuario	2 years ago
.gitignore	Cambios a ObjGraficos	2 years ago
README.md	Cambio de imágenes en Readme	2 years ago

README.md

Proyecto Frontend en Java Swing.

Este Proyecto Frontend es el resultado que da seguimiento al curso Interfaz Gráfica en Java proporcionado por el grupo Semana de Ingenio y Diseño de la universidad Distrital de Colombia donde se usan una serie de técnicas para realizar GUI modernas y profesionales.

Algunas partes del código están inspiradas en otros proyectos, a continuación se deja enlaces de estos:

- <https://github.com/akashyap2013/Advanced-Login-UI>

About

Este es el resultado que da seguimiento al curso Interfaz Gráfica en Java proporcionado por el grupo Semana de Ingenio y Diseño de la universidad Distrital de Colombia donde se usan una serie de técnicas para realizar GUI modernas y profesionales.

Readme

21 stars

1 watching

2 forks

Releases

No releases published

Packages



Proyecto Frontend en Java Swing

<https://github.com/CrissUD/ProyectoFrontendJava>

¿Preguntas?

IMPORTANTE

RETO 1 + 2

CONCLUSIONES